Docker Swarm Student Handbook

Part 1 – Foundations, .NET Web API, and PostgreSQL

Learning Objectives

By the end of this section, students will be able to:

- Explain what Docker Swarm is
- Differentiate between Manager and Worker nodes
- Explain Services, Tasks, and Replicas
- Install Docker Desktop and .NET SDK
- Create a .NET 8 Web API from scratch
- Create and run PostgreSQL containers
- Connect a .NET Web API to PostgreSQL
- Verify database connectivity

Chapter 1 – Introduction to Docker Swarm

What Problem Does Docker Swarm Solve?

Suppose you have a Web API running on one server.

Server A

|

+-- Web API

Everything works fine.

Then:

- More users start using the application.
- CPU usage increases.
- Memory usage increases.
- The server crashes.

Your application becomes unavailable.

This is called a:

Single Point of Failure

Traditional Solution

Buy a larger server.

Small Server

|

Large Server

This works temporarily.

Eventually the large server also reaches its limit.

Better Solution

Run the application on multiple servers.

Server A

Server B

Server C

Now we need a way to manage all servers.

Docker Swarm solves this problem.

What is Docker Swarm?

Docker Swarm is a clustering and orchestration tool built into Docker.

It allows multiple Docker hosts to behave like one system.

Instead of managing:

Server A

Server B

Server C

You manage:

Docker Swarm Cluster

Restaurant Example

Imagine a restaurant.

Without Swarm

Customer

|

Chef

One chef does everything.

If the chef leaves:

Restaurant Closed

With Swarm

Head Chef

|

| | |

Chef1 Chef2 Chef3

Work is distributed.

If Chef2 leaves:

Head Chef

|

| |

Chef1 Chef3

The restaurant continues operating.

Swarm Components

Manager Node

The brain of the cluster.

Responsibilities:

- Scheduling containers
- Managing workers
- Monitoring health

- Deploying services

Example:

Manager

Worker Node

Runs containers.

Example:

Worker 1

Worker 2

Worker 3

Workers do not make decisions.

Service

A service defines:

What should run

Example:

Web API

Replicas = 3

Swarm creates the containers automatically.

Task

A task is a running container created by a service.

Example:

Service

|

+-- Task 1

+-- Task 2

+-- Task 3

Replica

A copy of a service.

Example:

Web API

|

+-- Replica 1

+-- Replica 2

+-- Replica 3

Benefits:

- High availability
- Better performance
- Load balancing

Advantages of Docker Swarm

High Availability

If one node crashes:

Node A X

Swarm starts containers elsewhere.

Self Healing

Failed containers restart automatically.

Load Balancing

Traffic is distributed automatically.

Rolling Updates

New versions can be deployed gradually.

Chapter 2 – Environment Setup

Installing Docker Desktop

Download Docker Desktop.

Install with default settings.

Verify installation:

docker --version

Expected:

Docker version xx.xx.xx

Verify Docker Engine:

docker info

Expected output:

Containers: 0

Images: 0

Server Version: xx.xx

Installing .NET 8 SDK

Verify installation:

dotnet --version

Expected:

8.0.xxx

Verify Everything

Run:

docker --version

docker compose version

dotnet --version

All commands should return versions.

Chapter 3 – Creating a .NET Web API

Create Solution Folder

mkdir swarm-training

cd swarm-training

Create Web API

dotnet new webapi -n SwarmWebApi

Expected:

The template "ASP.NET Core Web API" was created successfully.

Open Project

cd SwarmWebApi

Run Application

dotnet run

Expected:

Now listening on:

http://localhost:5000

Open browser:

http://localhost:5000/weatherforecast

You should see JSON data.

Understanding Project Structure

SwarmWebApi

|

+-- Controllers

|

+-- Properties

|

+-- appsettings.json

|

+-- Program.cs

|

+-- SwarmWebApi.csproj

Simplifying Program.cs

Replace Program.cs with:

```
var builder = WebApplication.CreateBuilder(args);

var app = builder.Build();

app.MapGet("/", () =>
{
    return "Docker Swarm Lab API";
});

app.Run();
```

Run Again

dotnet run

Browse:

http://localhost:5000

Expected:

Docker Swarm Lab API

Chapter 4 – PostgreSQL Fundamentals

What is PostgreSQL?

PostgreSQL is a relational database.

It stores:

- Customers
- Orders
- Products
- Employees

in tables.

Database Example

Table:

Employees

Id	Name
1	John
2	Mary

Running PostgreSQL in Docker

Pull image:

```
docker pull postgres:16
```

Start PostgreSQL

```
docker run -d \
--name postgresdb \
-e POSTGRES_DB=appdb \
-e POSTGRES_USER=postgres \
-e POSTGRES_PASSWORD=postgres \
-p 5432:5432 \
postgres:16
```

Verify Container

docker ps

Expected:

postgresdb

Status:

Up

View Logs

docker logs postgresdb

Expected:

database system is ready to accept connections

PostgreSQL Connection String

Host=localhost;

Port=5432;

Database=appdb;

Username=postgres;

Password=postgres

Chapter 5 – Connecting .NET to PostgreSQL

Install Npgsql

Inside project:

dotnet add package Npgsql

Update Program.cs

using Npgsql;

var builder = WebApplication.CreateBuilder(args);

```
var app = builder.Build();

string connectionString =
"Host=localhost;Port=5432;Database=appdb;Username=postgres;Password=postgres";

app.MapGet("/", () =>
{
    return "API Running";
});

app.MapGet("/db", async () =>
{
    try
    {
        await using var connection =
            new NpgsqlConnection(connectionString);

        await connection.OpenAsync();

        var command =
            new NpgsqlCommand("SELECT NOW()", connection);

        var result =
            await command.ExecuteScalarAsync();

        return Results.Ok(result);
    }
    catch(Exception ex)
```

```
    {
        return Results.Problem(ex.Message);
    }
});

app.Run();
```

Run Application

dotnet run

Test Database Connection

Browse:

http://localhost:5000/db

Expected:

"2026-06-19T10:45:22..."

This confirms:

.NET Web API

|

|

PostgreSQL

communication is working successfully.

Knowledge Check

1. What is Docker Swarm?
2. What is a Manager Node?
3. What is a Worker Node?
4. What is a Service?
5. What is a Replica?
6. Why do we use PostgreSQL?

7. What package allows .NET to connect to PostgreSQL?
8. What command creates a Web API project?
9. What command starts PostgreSQL?
10. What endpoint verifies database connectivity?

Part 1 Summary

You have successfully:

Understood Docker Swarm fundamentals

Installed Docker and .NET

Created a .NET 8 Web API

Started PostgreSQL in Docker

Connected .NET to PostgreSQL